IBM Full Stack — JavaScript, TypeScript & Express.js

Understandable MCQ Master Notes

Prepared from the topics supplied for your IBM training/exam. Focus: fundamentals, rules, output traps, common differences and high-yield MCQ facts.

PART I — JAVASCRIPT

1. Basics

What JavaScript is

JavaScript is a dynamically typed language used in browsers and also on servers through Node.js. It is prototype-based and supports object-oriented, functional and event-driven programming.

Variables

var is function-scoped and can be redeclared. let and const are block-scoped and cannot be redeclared in the same scope. const prevents reassignment of the binding, but referenced objects/arrays can still be mutated.

Primitive values

string, number, bigint, boolean, undefined, symbol and null. Other values are objects/reference values.

Important traps

typeof null is 'object'. typeof undefined is 'undefined'. NaN has type number and is not equal to itself. Use Number.isNaN().

Hoisting

var declarations are hoisted and initialized to undefined. let/const are hoisted but remain in the Temporal Dead Zone until their declaration is reached.

2. Operators

Arithmetic

+, -, *, /, %, **. + also performs string concatenation when appropriate.

Comparison

== performs loose equality with coercion; === performs strict equality without coercion. Prefer === and !==.

Logical

&& returns the first falsy operand or the last operand; || returns the first truthy operand or the last operand; ?? uses the right side only for null or undefined.

Other operators

typeof, instanceof, in, delete, optional chaining ?., nullish coalescing ?? and ternary ?:.

3. Control Flow

Statements

if/else, switch, for, while, do...while, for...of and for...in.

for...of vs for...in

for...of iterates values of an iterable such as an array. for...in iterates enumerable property keys.

break/continue/return

break exits a loop or switch; continue skips the current iteration; return exits the current function.

Truthiness

Falsy values include false, 0, -0, 0n, '', null, undefined and NaN. [] and {} are truthy.

4. Objects

Object basics

Objects store key-value pairs. Access with obj.name or obj['name'].

Methods and this

A function stored as an object property is a method. this depends on the call site for normal functions; arrow functions use lexical this.

Destructuring

const {name}=user extracts a property. const [a,b]=arr extracts array elements.

Spread/rest

{...obj} creates a shallow copy of enumerable own properties. Rest collects remaining properties/arguments.

Equality trap

Object === compares references, not structural content.

5. Arrays

Basics

Arrays are objects with indexed elements and a length property. They can contain mixed types.

High-yield methods

push/pop change the end; shift/unshift change the beginning; slice returns a shallow copy without mutating; splice mutates; map transforms; filter selects; reduce accumulates; find returns the first match; some/every test conditions.

sort trap

sort() compares as strings by default. For numbers use (a,b)=>a-b.

Reference trap

const b=a; b.push(3) changes a too because both variables reference the same array.

6. Functions

Types

Function declarations, expressions, arrow functions, callbacks, higher-order functions and IIFEs are common.

Arrow functions

Concise syntax and lexical this. They do not have their own arguments, this, super or new.target.

Default/rest

function f(x=10) uses 10 when x is undefined. ...args collects remaining arguments into an array.

Callback

A callback is a function passed to another function. map/filter/reduce commonly receive callbacks.

Closure

A function can remember variables from its lexical outer scope even after the outer function returns.

MCQ trap

Function declaration hoisting differs from function-expression variables. let/const function expressions are inaccessible in the TDZ before initialization.

PART II — TYPESCRIPT

1. Getting Started

What TypeScript is

TypeScript is a statically typed superset of JavaScript. TypeScript source is compiled/transpiled to JavaScript.

Benefits

Static checking catches many errors before runtime and improves autocomplete, refactoring and maintainability.

Key point

Most TypeScript types are compile-time information; normal JavaScript runtime does not automatically enforce type annotations.

2. TypeScript Basics & Basic Types

Common types

string, number, boolean, bigint, symbol, any, unknown, void, null, undefined, never, object. Arrays can be T[] or Array<T>.

any vs unknown

any disables most type checking. unknown is safer because it must be narrowed before use as a specific type.

Union/intersection

A | B means either type. A & B combines requirements from both types.

Inference

TypeScript often infers types automatically; explicit annotations are useful when they clarify or constrain APIs.

Literal types

A type such as 'left'|'right' permits only those literal values.

3. Compiler & Configuration

tsc

The TypeScript compiler checks and emits JavaScript. tsc --init creates tsconfig.json.

Important options

target controls emitted JS version; module controls module system; strict enables strong checking; outDir sets output folder; rootDir describes source root; sourceMap generates source maps.

Strict mode

strict enables important checks such as strictNullChecks and noImplicitAny.

Trap

Compilation success does not guarantee runtime correctness; most types are erased from emitted JavaScript.

4. Next-generation JavaScript & TypeScript

Modern JS

let/const, template literals, destructuring, spread/rest, classes, modules, promises, async/await, optional chaining and nullish coalescing.

TypeScript additions

Interfaces, type aliases, access modifiers, generics, enums, decorators depending on version/configuration, utility types and advanced narrowing.

Async

async functions return Promises. await pauses that async function until settlement; it does not block the whole JS runtime.

5. Classes & Interfaces

Classes

Support constructors, properties, methods, inheritance and public/private/protected access modifiers.

Interfaces

Describe a shape/contract. A class can implement multiple interfaces. Interfaces are normally erased from JS output.

extends vs implements

A class extends one class but can implement multiple interfaces. An interface can extend multiple interfaces.

readonly

A readonly property cannot normally be reassigned after initialization through the typed object.

Private trap

TypeScript private is mainly compile-time access control. ECMAScript #private fields provide runtime-enforced private fields.

6. Advanced Types

Narrowing

typeof, instanceof, in and user-defined type predicates can narrow union types.

Discriminated unions

A shared literal property such as kind can safely identify which union member is present.

keyof

keyof T produces a union of property keys of T.

typeof in types

typeof can derive a type from an existing value in a type position.

Utility types

Partial, Required, Readonly, Pick, Omit, Record, Exclude, Extract, NonNullable and ReturnType are high-yield.

never

Represents an impossible/no-return value and is useful for exhaustive checking.

7. Generics

Purpose

Generics preserve type information in reusable code: function identity<T>(x:T):T.

Constraints

T extends SomeType limits the allowed type argument.

Generic structures

Generic classes and interfaces such as Box<T> and ApiResponse<T> are common.

Trap

Generics are compile-time constructs; they do not automatically create runtime type checks.

8. Decorators

Concept

Decorators attach metadata or behavior to classes/members depending on the decorator model and TypeScript configuration.

Version caution

Traditional experimentalDecorators syntax and newer standard decorators have different semantics. MCQs can depend on the TypeScript version/configuration used by the curriculum.

Exam tip

Know whether the question refers to legacy decorators or the newer standard decorator model before applying an ordering rule.

9. Drag & Drop Project

Browser concepts

dragstart, dragover and drop are common events. dragover usually needs preventDefault() to allow dropping.

Type safety

Interfaces/classes can model draggable items and callbacks.

DOM nullability

document.querySelector can return null; strict TypeScript may require null checks or narrowing.

10. Modules & Namespaces

Modules

export exposes declarations and import consumes them. ES modules are file-based and scoped.

Default vs named

Default exports are imported without braces and can be renamed; named exports use braces and can be aliased.

Namespaces

Namespaces group declarations but modern applications generally prefer ES modules.

Trap

A file containing top-level import/export is treated as a module.

11. Webpack with TypeScript

Webpack

Webpack bundles modules and dependencies into output assets.

Typical flow

TypeScript source → configured loader/transpiler such as ts-loader → bundled JavaScript → browser.

Configuration

entry is the starting module; output defines bundle; module.rules define processing; plugins extend build behavior.

Trap

Webpack is a bundler, not the TypeScript compiler itself.

12. Select & Share a Place / Google Maps

Typical flow

Collect input → validate/convert → call map/geocoding API → display result → share/store place.

Type safety

Interfaces can describe coordinates and API responses; handle null/undefined and failed requests.

API keys

Use the provider's recommended restrictions and never expose secrets that are intended to remain server-side.

MCQ focus

Promises/async-await, DOM events, modules, classes/interfaces and typed API responses are the reusable concepts.

PART III — EXPRESS.JS

1. Introduction

What Express is

Express is a minimal web framework for Node.js used to build HTTP servers, web applications and APIs.

Core idea

An Express application is a middleware pipeline: requests pass through middleware and route handlers.

Basic server

const express=require('express'); const app=express(); app.listen(3000);

Trap

Express runs on Node.js; it is not a browser framework.

2. Environment Setup

Typical setup

Install Node.js, create a project, run npm init, then install Express. package.json stores metadata, scripts and dependencies.

Dependencies

npm install express adds a runtime dependency; npm install -D adds a development dependency.

Scripts

npm scripts provide repeatable commands such as npm start or npm run dev.

Trap

node_modules is normally not committed; package.json/package-lock.json describe dependencies.

3. Req & Res, Router & Generator

Request

req contains request data such as params, query, headers and body.

Response

res sends data with methods such as res.send(), res.json(), res.status() and res.redirect().

params/query/body

/users/:id gives req.params.id. /users?id=5 gives req.query.id. JSON body data is available after express.json().

Router

express.Router() creates modular routes that can be mounted with app.use('/users', router).

Generator

Express Generator scaffolds a project; it is a generator, not Express itself.

Middleware trap

Order matters. Middleware must call next() or end the response, otherwise the request can hang.

4. Movie Fan App

MVC idea

Model = data/domain; View = presentation; Controller = request/response logic.

Static files

express.static(...) serves static assets.

Error handling

Express error middleware has four parameters: (err, req, res, next). Register it after normal routes/middleware.

5. Building an API

REST basics

GET retrieves, POST creates/submits, PUT generally replaces, PATCH partially updates, DELETE removes.

Status codes

200 OK; 201 Created; 204 No Content; 400 Bad Request; 401 authentication required; 403 Forbidden; 404 Not Found; 409 Conflict; 500 Internal Server Error.

JSON

app.use(express.json()) parses JSON request bodies; res.json(obj) sends JSON.

Validation/security

Validate inputs, use appropriate status codes, authenticate/authorize protected resources, avoid leaking internal errors and use safe DB access.

6. Passport

Purpose

Passport is authentication middleware for Node.js/Express and supports multiple authentication strategies.

Flow

A strategy verifies credentials/user, then authentication state is established using a session or another mechanism.

Sessions

With session authentication, serializeUser stores user identity and deserializeUser retrieves the user on later requests.

Trap

Authentication asks who you are; authorization asks what you are allowed to do.

7. Database Connection

Connection lifecycle

Configure a connection/pool, handle connection errors, execute operations asynchronously and close resources during shutdown.

Environment variables

Keep credentials and connection strings in environment/configuration rather than hard-coding them.

Async DB

Most Node.js database drivers use callbacks or Promises; async/await is common with Promise APIs.

Security

Validate input and use parameterized queries/ORM mechanisms to reduce injection risk. Never send DB credentials to clients.

FINAL — RAPID MCQ REVISION SHEET

Concept	Must remember
var / let / const	var=function scope; let/const=block scope; const binding cannot be reassigned
== / ===	== allows coercion; === strict equality
typeof null	'object'
NaN	type number; NaN !== NaN
slice / splice	slice does not mutate; splice mutates
for...of / for...in	values / enumerable keys
Arrow function	lexical this; no own arguments
Closure	function retains access to lexical outer variables
any / unknown	any weakens checking; unknown requires narrowing
union / intersection	A|B either; A&B combined requirements
extends / implements	inheritance / contract implementation
keyof	property-key union of a type
tsconfig target	emitted JS target
Webpack	module bundler
Express	Node.js web framework
req.params	route parameters
req.query	query-string parameters
req.body	request body after parser middleware
next()	passes control to next middleware
GET/POST/PATCH/DELETE	read/create/partial update/delete
201	Created
401 / 403	authentication required / forbidden
Passport	authentication middleware
Authentication / authorization	identity / permissions

IBM MCQ Strategy

Prioritize differences and traps: var/let/const, ==/===, slice/splice, for...of/for...in, any/unknown, interface/type, extends/implements, Express req.params/query/body, middleware order, REST methods, status codes and authentication vs authorization. For output questions, trace scope, hoisting, this, mutation and async behavior.